Resume Project Interview Guide

Code-grounded study guide for personal projects on Sheel Patel's resume

Scope. This guide covers the two entries under **Projects** in the supplied resume, not the work-experience entries:

1. **LLM Gateway** — Java, Spring Boot, Redis, PostgreSQL/pgvector, Docker, Prometheus.
2. **Polyglot Quant & AI Microservices** — Java/Spring Boot, Python/FastAPI, LangChain4j, PostgreSQL, Docker.

How to use this guide. First master each project's 60-second story, then its request flow, then the “Questions to rehearse” sections. Every claim labelled **implemented** was verified in the current workspace code. Items labelled **gap / next step** are intentional interview talking points: explain the trade-off honestly and say how you would harden the design.

Important accuracy rule. Do not claim features that are only described in a README. In particular, the gateway does not currently fail over after an upstream HTTP failure, and the polyglot system's production FastAPI path does not use its newer local scoring engine.

Table of Contents

1. Interview strategy and project comparison
2. LLM Gateway: architecture, flow, internals, questions
3. Polyglot Quant & AI Microservices: architecture, flow, quant math, questions
4. Cross-project system-design questions
5. Rapid-review answer sheet and source map

1. Interview Strategy and Project Comparison

1.1 A reliable answer shape

For any project question, answer in this order:

1. **Problem and boundary:** what client problem the system solves and what the system owns.
2. **Architecture:** name the services, stores, and synchronous/asynchronous boundaries.
3. **Critical path:** trace one request with concrete decisions.
4. **Trade-offs:** why the chosen mechanism fits; what it costs.
5. **Reliability/security/testing:** state what is implemented and one improvement you would prioritize.

Avoid a technology inventory ("I used Redis, Docker, Postgres...") without connecting each tool to a design requirement.

1.2 Comparison at a glance

Dimension	LLM Gateway	Polyglot Quant & AI Microservices
Primary purpose	One canonical chat API in front of multiple LLM vendors	Combine deterministic market signals with an LLM-driven stock-analysis workflow
Entry point	`POST /v1/chat/completions`	`GET /agent/analyze/{ticker}` and `GET /api/v1/score/{ticker}`
Main orchestration language	Java 21 / Spring Boot	Java 17 / Spring Boot, calling Python 3.12 / FastAPI
Fast state	Redis exact-response cache and ZSET limiter	JVM-local conversation memory only
Durable state	PostgreSQL, pgvector semantic cache, API-key schema	PostgreSQL JPA entities for users, portfolios, recommendations
Core algorithm	SHA-256 canonical cache key; cosine similarity; atomic Redis sliding window	Trend/momentum composite score from SMA, MACD, RSI, and CCI
Resilience design	Cache-first, provider selection, timeout on adapter calls	Quant-service retry and degraded prompt injection; agent/evaluation retries
Important current gap	Upstream provider failures do not trigger fallback	Divergent quant engines; thin automated test coverage

1.3 60-second overview: LLM Gateway

"I built a Spring Boot API gateway that gives a client one chat-completion contract while adapters translate requests to OpenAI, Anthropic, or Gemini. The request path is cache-first: a SHA-256 key over a canonical request checks Redis, then an optional pgvector semantic lookup uses cosine similarity, and only a miss reaches a provider. At the edge I added HMAC-hashed API keys and an atomic Redis ZSET/Lua sliding-window limiter. PostgreSQL stores API keys and semantic-cache records, and Prometheus exposes cache, authentication, and rate-limit counters. I tested the integrations with Testcontainers and WireMock. The next production investment would be true runtime health checks and a circuit breaker/retry policy around provider calls, because the current routing fallback handles a missing configured key rather than a live provider outage."

1.4 60-second overview: Polyglot system

"I separated deterministic quantitative computation from LLM orchestration. A Python FastAPI service fetches weekly OHLC data and produces a composite technical score. A Java Spring service calls that service over the Docker network, injects the

score into a LangChain4j prompt, lets the agent use market-data tools, evaluates the structured recommendation, and persists it with JPA. The split lets Python own the data-science ecosystem and Java own orchestration, persistence, and the REST surface. If the quant service is unavailable, Java passes an explicit unavailable payload to the prompt and continues with external tools instead of failing the entire pipeline. The main technical debt is unifying the three scoring implementations and adding contract/integration tests before treating the service as production-ready.”

2. LLM Gateway

Workspace: `llm-gateway/`

Core stack: Java 21, Spring Boot 3.3.5, JDBC/Flyway, Redis, PostgreSQL + pgvector, WebClient, Micrometer/Prometheus, Docker Compose, Testcontainers, WireMock.

2.1 Problem, contract, and boundaries

The gateway hides vendor-specific chat APIs behind one client contract. It owns request validation, API-key authentication, rate limiting, cache policy, model routing, response normalization, telemetry, and selected persistence. It does **not** own an LLM model, streaming protocol, quota enforcement, or a true circuit breaker.

The canonical entry point is:

```
POST /v1/chat/completions
Content-Type: application/json
X-API-Key: <optional unless gateway auth is enabled>

{ model, messages: [{ role, content }], temperature?, maxTokens?, userId? }
```

The controller validates nonblank `model` and nonempty `messages`, calls one service method, and returns a normalized `ChatResponse`. Provider adapters handle incompatible wire formats behind the `LlmProvider` interface.

2.2 Architecture

```
Client
  |
  v
GatewaySecurityFilter ----------------------> ApiKeyService -> PostgreSQL api_keys
  |                                             |
  |                                             v
  |-----------------------------------------> Redis ZSET + atomic Lua limiter
  v
ChatController -> ChatCompletionService
  |                  |                |
  |                  |                +--> ExactResponseCache -> Redis String
  |                  |
  |                  +--> OpenAiEmbeddingClient -> SemanticResponseCache -> pgvector
  v
ProviderRouter -> OpenAI | Anthropic | Gemini adapters -> external vendor API
  |
  +--> normalized ChatResponse -> caches -> client

Metrics/logs -> Micrometer Prometheus endpoint and JSON stdout logs
```

Configuration is enabled by `config/GatewayConfig.java`; `application.yml` binds LLM-provider, cache, and gateway-security properties. The package boundary is intentional:

Package	Responsibility
`api`	HTTP controllers, request/response DTOs, validation, exception mapping
`service`	Cache-to-provider orchestration
`cache`	Request canonicalization, exact cache, embedding client, semantic cache
`provider`	Vendor adapters, model translation, selection

Package	Responsibility
`security`	API-key generation, HMAC hashing, authentication filter, repository
`ratelimit`	Redis Lua sliding-window implementation
`observability`	Provider health actuator contribution

2.3 Full request walkthrough

Step 1: security filter

`GatewaySecurityFilter` runs on `/v1/*` except `/v1/admin/*`, `/health`, `/actuator/*`, and Swagger paths. Authentication and rate limiting are configurable and default to disabled for local development.

When auth is enabled:

1. Read the configured header, normally `X-API-Key`.
2. HMAC-SHA256 the presented secret.
3. Query the `api_keys` table for a non-revoked row with that hash.
4. Attach `apiKeyId` and `userId` request attributes.
5. Choose a rate-limit bucket: `llm:rl:v1:{apiKeyId}` or anonymous client IP.

On an allowed request, the response gets `X-RateLimit-*` headers. A rejected request gets HTTP 429 and `Retry-After`.

Step 2: exact cache

`ChatCompletionService.complete()` first derives a deterministic key:

```
llm:exact:v1:SHA256(canonical JSON)
```

The canonical JSON includes `model`, each message's `role` and `content`, and optional `temperature`, `maxTokens`, and `userId`. Object key order cannot cause a false miss. A cache hit returns immediately with `fromCache=true`; Redis `GET` is expected O(1) average time. Hashing is O(n) in request size.

Step 3: semantic cache

On an exact miss, and only when semantic caching and an OpenAI key are enabled:

1. Concatenate role/content messages and model into an embedding input.
2. Call OpenAI's embedding model (`text-embedding-3-small` by default).
3. Query PostgreSQL for the nearest unexpired row for the same model:

```
SELECT id, response_json
FROM semantic_cache
WHERE expires_at > now()
  AND model = ?
  AND (1 - (embedding <=> ?::vector)) >= ?
ORDER BY embedding <=> ?::vector
LIMIT 1;
```

<=> is pgvector cosine distance, so similarity is `1 - distance`. The default minimum similarity is 0.95. The migration creates an IVFFlat cosine index with 100 lists, which favors approximate-nearest-neighbor speed at scale over exact exhaustive search.

Trade-off to explain: semantic hits can serve paraphrases and reduce an expensive chat call, but the current implementation still needs an embedding API call on every semantic-eligible lookup. Its embedding text excludes `temperature`, `maxTokens`, and `userId`, so a semantic hit can be less precise than exact caching. The cache is a performance optimization, not a source of truth.

Step 4: provider selection and normalization

`ProviderRouter` selects the first healthy provider that supports the requested model. The adapters support:

Adapter	Recognized models	Upstream shape
OpenAI	`gpt-*`, `o1*`, `o3*`, `ft:*`	`/v1/chat/completions`, Bearer token
Anthropic	`claude-*`	`/v1/messages`, `x-api-key`
Gemini	`gemini-*`	`/v1beta/models/{model}:generateContent?key=...`

The adapters translate roles and request fields and map vendor responses back to the gateway DTO. For example, Anthropic merges system messages and uses a default `max_tokens` of 1024; Gemini maps the assistant role to `model`.

Precise limitation: provider “health” is currently API-key presence, not a probe. If the selected provider returns a timeout, 429, or 5xx, its `ProviderException` becomes a gateway error rather than causing a second provider attempt. Model translation has mappings for a limited set of cross-vendor models. Say “health-based selection for configured providers,” not “outage-tested active failover.”

Step 5: write caches and return

Live responses always populate the Redis exact cache. If an embedding was available, the response is inserted in `semantic_cache`. Inserts are not deduplicated/upserted; expired semantic rows are filtered at query time.

2.4 Rate limiting: why ZSET plus Lua

A fixed window permits burstiness around a boundary. A sliding window uses actual request timestamps.

The Lua script in `RedisSlidingWindowRateLimiter` executes these operations atomically on one Redis server:

```
cutoff = nowMs - windowMs
ZREMRANGEBYSCORE key 0 cutoff   # evict old timestamps
count = ZCARD key
if count >= limit: return blocked
ZADD key nowMs "{nowMs}-{uuid}" # unique members avoid same-millisecond collision
EXPIRE key windowSeconds + 2
return allowed
```

This prevents the race present in `GET count` followed by a separate `INCR`/write: concurrent clients cannot all see `limit - 1` and then all increment. Complexity is:

Operation	Complexity
Remove expired timestamps	O(log N + M), M removed members
Count	O(1)

Operation	Complexity
Add current request	O(log N)

N remains bounded by the cap in steady state. The property names say RPM, but technically it is “requests per configured window”; it behaves as requests per minute only with the default 60-second window.

2.5 Security design

API-key lifecycle

1. Admin creates an `lgw_` key from 32 cryptographically random bytes encoded URL-safe Base64.
2. The plaintext is returned only at creation.
3. The application stores `HMAC-SHA256(plaintext, server secret)` in `api_keys.key_hash`.
4. Incoming credentials are HMACed the same way and equality-queried.
5. Revocation sets `revoked_at`; future lookups exclude it.

HMAC is a deliberate lookup trade-off: it supports fast deterministic lookup, unlike a salted password hash that requires scanning candidate hashes. The trade-off is the HMAC secret becomes critical—if it is compromised, an attacker can test guessed keys offline. For a production deployment, place it in a secret manager, rotate it carefully, terminate TLS upstream, audit admin calls, and use constant-time comparison where direct secret comparison is unavoidable.

Admin boundary and limitations

`/v1/admin/api-keys` is protected by an `X-Admin-Token` equality check, and is intentionally excluded from the normal API-key filter. This is practical for a demo but should evolve toward role-based admin identity, TLS, rate limiting, audit logging, and secret rotation. There is no Spring Security filter chain.

2.6 Persistence and caching model

Flyway migrations create:

Table	Why it exists	Important fields/indexes
`api_keys`	API identity and per-key limit configuration	unique `key_hash`, `rate_limit`, `revoked_at`, `monthly_budget_usd`
`usage_records`	Intended cost/usage audit record	provider/model/tokens/cost/cache hit/latency; time indexes
`semantic_cache`	Durable approximate response reuse	1536-d `vector`, prompt/request/response JSON, TTL, model, IVFFlat index

Honest gap: `usage_records` and `monthly_budget_usd` are schema-only in the current code. Do not claim budget enforcement or cost analytics. A correct next implementation would write a usage row after a live provider response, normalize usage fields across adapters, estimate cost from a versioned pricing table, and enforce a budget transactionally or with an atomic spend reservation.

2.7 Observability and deployment

The service emits Logstash-style JSON logs to stdout and custom Micrometer counters:

Metric	Useful question it answers
`llm.cache.exact` / `llm.cache.semantic` with result tags	Are caches useful or failing?
`llm.gateway.auth`	Are invalid/missing credentials increasing?
`llm.gateway.ratelimit`	Is a client being throttled?

`/actuator/prometheus` is scraped by Prometheus in Compose; Grafana is included as a container. `/health` is only a liveness endpoint, while the actuator health contributor reports configured-provider status. Neither is a complete readiness gate for real upstream availability.

Compose runs application, PostgreSQL pgvector, Redis, Prometheus, and Grafana. The app waits for Redis/PostgreSQL health checks. The image is a multi-stage build: Gradle/JDK 21 produces the jar, then a Temurin JRE Alpine runtime executes it. Container builds skip tests, so CI should run tests separately before image publication.

2.8 Testing: what is actually proven

Test	What it proves
`ProviderRouterTest`	supported-model selection and fallback when a provider reports unhealthy
`ChatCompletionIntegrationTest`	vendor request mapping, one upstream call across exact cache hit, liveness
`SemanticCacheIntegrationTest`	semantic lookup can skip the second chat call; embeddings still occur
`AuthRateLimitIntegrationTest`	API-key creation, `whoami`, and a 429 after limit exhaustion

Integration tests use Testcontainers for Redis/PostgreSQL and WireMock for upstream LLM simulation. They are disabled without Docker. Missing coverage includes provider HTTP-error failover, concurrent limiter pressure, revocation, cost recording, and circuit-breaker behavior.

2.9 Questions to rehearse

Why use two caches instead of only Redis?

Exact cache is deterministic, cheap, and has no semantic false-positive risk. Semantic cache trades some precision for reuse when requests are paraphrases. Redis is ideal for low-latency TTL key/value responses; pgvector keeps embeddings and response JSON queryable durably using similarity search. I would instrument hit quality and segment cache policy by model and task before allowing semantic reuse for sensitive workflows.

How does the Lua script avoid a race condition?

Redis runs a Lua script atomically. Removing stale events, checking the count, adding the timestamp, and refreshing TTL happen as one operation. No other client can interleave a request between count and add.

What happens when two identical cache misses arrive at once?

Both can miss and call the vendor because the current code does not implement request coalescing/singleflight. The next improvement is a short-lived per-key distributed lock or in-process singleflight, with careful timeout and leader-failure handling.

Why use HMAC-hashed API keys?

The system needs a deterministic database lookup without retaining plaintext keys. HMAC makes it possible to hash the presented value and find the row directly. The trade-off is protecting and rotating the server HMAC secret.

How would you implement real failover?

Wrap provider calls with timeouts, bounded retries only for safe/transient classes, circuit breakers, and a provider-health model driven by success/error/latency windows. On a retriable failure, translate the requested model only through approved quality/cost-compatible fallback mappings. Record the selected provider and reason, and avoid retries for invalid client requests.

Why is WebClient used with `.block()`?

WebClient provides a flexible HTTP client, but the current MVC service uses blocking calls with a two-minute timeout. For high concurrency, I would either use a blocking HTTP client with a bounded worker pool or make the controller/service reactive end-to-end; mixing paradigms can waste event-loop resources.

2.10 LLM Gateway “implemented vs next” summary

Implemented	Next production step
Unified vendor API, three adapters, normalized DTO	Runtime health checks, retries, circuit breakers, outage failover
Redis exact cache	Singleflight and cache invalidation/policy controls
pgvector semantic cache	Avoid unnecessary embeddings; dedupe/expiry job; evaluate quality
HMAC key storage and revocation	RBAC/admin audit/TLS/secret rotation
Atomic sliding window	Distributed policy/config and abuse analytics
Prometheus counters, structured logs	Tracing, SLOs, dashboards, alert rules
Testcontainers/WireMock integration tests	Concurrency, error-path, security, and load tests

3. Polyglot Quant & AI Microservices

Workspace: `polyglot-system/`

Services: Finance Vibe Python quant scorer, FinSight AI Java/LangChain4j orchestrator, PostgreSQL; optional React dashboard.

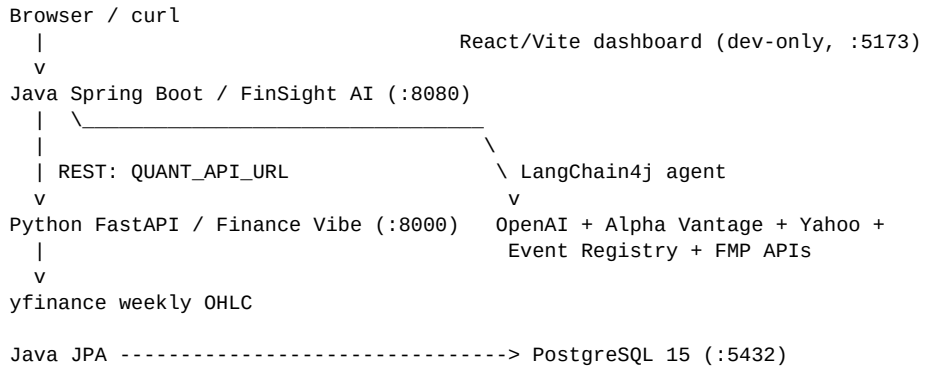
3.1 Problem and reason for the language split

The system combines two different workloads:

- **Python service:** market-data retrieval and technical-indicator computation. Python has natural libraries for `yfinance`, pandas, NumPy-style numerical operations, and `pandas_ta`.
- **Java service:** web endpoints, dependency injection, typed JPA persistence, orchestration, validation, and LangChain4j integration.

The boundary is REST over the private Docker network. It avoids embedding Python computation into Java and lets either service scale or evolve independently. The cost is distributed-systems complexity: network failures, versioned contracts, observability across services, and duplicated deployment configuration.

3.2 Deployed architecture



`docker-compose.yml` starts Python, Java, and PostgreSQL. Java receives `QUANT_API_URL=http://python-scoring-service:8000` and waits for Python and PostgreSQL health checks. The React application is not part of Compose; it runs separately through Vite. Python and PostgreSQL have health checks; Java does not currently declare one.

3.3 Main request flow

The code has two Java agent paths. For this project narrative, use the primary polyglot path:

```
GET /agent/analyze/{ticker}
  -> AgentController
  -> AgentService.analyzeAndSaveWithValidation()
  -> retry outer analysis attempt
  -> AgentService.runFullAnalysis()
       -> build in-memory context
       -> QuantitativeScoringClient.fetchScore(ticker)
       -> format memory + quant JSON + ticker into prompt
       -> LangChain4j assistant calls selected tools
```

```
        -> evaluator judges structured draft
        -> up to two repair prompts if fixable
  -> StockRecommendationValidator
  -> JPA save StockRecommendation
  -> response
```

`GET /api/agent/market/{ticker}` is a legacy/simple path that calls a model directly and does not fetch the Python quant score. Distinguish it if asked.

3.4 Python scoring API and error handling

Finance Vibe exposes:

Method	Path	Behavior
GET	`/health`	Returns `{"status":"ok"}`
GET	`/api/v1/score/{ticker}`	Validates ticker, downloads data, calculates and returns score/signals

The endpoint strips/uppercases the ticker, rejects an empty ticker or one longer than 10 characters with 400, and returns 500 for unexpected computation/data errors. `compute_vibe_score()` downloads ten years of weekly candles with `auto_adjust=True`, flattens Yahoo's possible MultiIndex columns, and drops a partial current week when it is not Friday.

The returned contract includes ticker, timestamp, period, interval, composite score, and raw signal values:

```
{
  "ticker": "AAPL",
  "asOf": "UTC ISO timestamp",
  "period": "10y",
  "interval": "1wk",
  "score": 4.0,
  "signals": {
    "close": 0, "sma20": 0, "sma50": 0,
    "macdHist": 0, "macdHistSignal": 0,
    "rsi": 0, "rsiSignal": 0,
    "cci": 0, "cciSignal": 0
  }
}
```

The response is numerical input for the LLM, not a trading recommendation. It should be framed as educational/demo software; historical technical signals do not guarantee future performance.

3.5 Quantitative scoring mathematics

The **production FastAPI route** imports `analysis_engine.py`. It uses a simple additive score:

```
Composite = Trend contribution + Momentum contribution + CCI contribution
Theoretical range: -10 through +10
```

Trend: simple moving averages

```
SMA20 = mean(Close over last 20 weekly candles)
SMA50 = mean(Close over last 50 weekly candles)
```

Condition	Contribution	Interpretation
Close > SMA20 > SMA50	+4	bullish price and medium-term trend alignment
Close > SMA20 only	+2	partial bullish signal
Close < SMA20 < SMA50	-4	bearish alignment
Otherwise	0	mixed/neutral regime

Momentum: MACD histogram and RSI

For the served API, MACD uses nonstandard `fast=15`, `slow=30`, `signal=9`. The implementation takes the MACD histogram and compares it to a 20-period EMA of that histogram. RSI is a `pandas_ta` 14-period RSI compared to its 20-period EMA.

Condition	Contribution
MACD histogram > histogram signal **and** RSI > RSI signal	+3
Exactly one condition true	+1
Neither true	-3

An interview-ready explanation: MACD captures the relation between fast and slow exponential trends; using its histogram emphasizes momentum change. RSI measures the balance of recent gains/losses. Comparing each to a smoothed signal reduces reaction to a single noisy point, though it adds lag.

CCI: typical price and mean absolute deviation

```
TP = (High + Low + Close) / 3
SMA_TP = mean(TP over 20 periods)
MAD_TP = mean(abs(TP_i - SMA_TP) over 20 periods)
CCI = (TP - SMA_TP) / (0.015 * MAD_TP)
```

Condition	Contribution
CCI > CCI signal and CCI > 0	+3
CCI < CCI signal and CCI < 0	-3
Otherwise	0

CCI normalizes how far current typical price is from its recent central tendency. `0.015` is the conventional scaling factor intended to make many values fall roughly in the ±100 region. The main FastAPI engine does **not** guard the zero-MAD case; flat price windows can yield NaN/infinity. A local shadow engine floors the denominator at `1e-9`, but it is not currently wired to the API.

3.6 Important scoring-engine discrepancy

Three similar modules exist:

Module	Used by FastAPI?	Key behavior
`analysis_engine.py`	Yes	15/30/9 MACD, EMA20 comparison, simple additive score; no CCI zero guard
`analysis_engine_local.py`	No; offline/batch comparison	12/26/9 MACD, different smoothing, robust denominator floor and richer logic
`signals.py`	No; legacy CLI	another indicator implementation

Be precise if an interviewer asks about “robust CCI,” 12/26/9 MACD, RSI caps, or persistence checks: those ideas exist in the local/shadow path and documentation, while the HTTP service currently runs the simpler implementation. A strong answer is: “I would unify the production endpoint on one tested indicator library/module, define a versioned score contract, add fixtures for known price sequences, and expose the scoring version in every response.”

There is also a documentation mismatch: config uses a 10-year weekly period even though parts of the README describe a five-year regime.

3.7 Java orchestration and LangChain4j

The main assistant, evaluator, and some tools use `gpt-4o-mini`. The main model temperature is 0.3, evaluator temperature is 0.0, and the main assistant keeps a `MessageWindowChatMemory` of at most 15 messages. That memory is an in-process `ArrayList`: it is lost on restart and not shared among replicas.

The main agent registers seven tools:

Tool	External source	Failure behavior / purpose
`PriceTool`	Alpha Vantage daily series	Returns market price data; can throw runtime exception
`TrendingTool`	Yahoo trending	Returns empty list on failure
`HistoryTool`	Yahoo chart API	Returns `{}` on failure
`SentimentTool`	LLM call	Classifies sentiment
`NewsTool`	Event Registry	Returns `[]` on failure
`PredictedGainTool`	Alpha Vantage	returns error signal on failure
`FundamentalSignalTool`	FMP metrics	Returns `{}` on failure

The prompt is filled with memory context, JSON from the Python quant endpoint, and the ticker. This is “dynamic prompt injection” in the ordinary engineering sense—server-supplied contextual data is composed into a template—not user-controlled prompt text being trusted as instructions.

Degraded mode and retries

`QuantitativeScoringClient` retries the Python call twice with 400 ms exponential backoff. If it remains unavailable, it returns a `QuantScoreResponse.unavailable()` object instead of throwing. The template tells the LLM to disclose the missing quant score and proceed using its tools.

The outer analysis retry runs twice with a 1-second delay for `RuntimeException`. Then an evaluation loop can issue up to two repair prompts. This is graceful degradation, but not a circuit breaker: the REST client has no explicit timeout, there is no bulkhead, and repeated pressure can still cascade.

Structured-output validation

The evaluator assesses a draft and can request repairs. `StockRecommendationValidator` performs final field validation before JPA persistence. A current mismatch deserves candid treatment: the validator requires `predictedGain`, whereas the prompt's output schema does not consistently include it and says “no numeric predicted gain” in one decision section. Fix the contract before relying on it: use one Java record/JSON schema for prompt, model parser, validator, API response, and database mapping; add an integration test with a representative model response.

3.8 Persistence model

JPA uses `spring.jpa.hibernate.ddl-auto=update`; there are no Flyway/Liquibase migrations. Entities include:

Entity	Purpose	Relationship / constraint
`StockRecommendation`	persisted agent output: ticker, action, horizon, confidence, price, reasoning	queried by ticker/horizon; `createdAt` is JSON-hidden
`User`	user financial profile	unique email
`PortfolioAsset`	holdings	many-to-one User
`Income`	income records	many-to-one User
`Expense`	spending records	many-to-one User

There are no explicit JPA indexes on `StockRecommendation.ticker` or `horizon`, so repeated filtering will eventually need indexes. `ddl-auto=update` is convenient locally but unsafe as production schema management: it lacks reviewed, reproducible, reversible migrations.

3.9 Frontend and API surface

The React 19/Vite/Tailwind dashboard stores results in browser `localStorage` under `finance_ai_results`. It calls `http://localhost:8080/agent`, uses trending/multi-select and manual ticker analysis, and renders action, horizon, confidence, and reasoning. It does not show all persisted fields such as predicted gain, price, or created time.

Important Java routes:

Base	Endpoint	Use
`/agent`	`GET /analyze/{ticker}`	primary full polyglot flow + save
`/agent`	`GET /trending`	Yahoo trending tickers
`/agent`	`GET /scan/trending`	batch analysis
`/agent`	`GET ?q=`	raw debug chat
`/api/agent/market`	`GET /{ticker}`	legacy non-polyglot agent
`/api/recommendations`	CRUD/filtering	recommendation data
`/api/users`, `/api/portfolio`, `/api/income`, `/api/expenses`	CRUD	financial profile data

3.10 Security and reliability posture

This is a development/demo architecture:

- No Spring Security dependency, authentication, authorization, or rate limiter.
- Compose uses default PostgreSQL credentials and publishes port 5432 to the host.
- Secrets come from environment files; production should use a managed secret store.
- `WebConfig` allows localhost Vite origin with credentials; another controller has a broader CORS annotation, so policy needs consolidation.
- Ticker validation exists in Python, but Java route validation and third-party tool input boundaries are limited.
- No queue, durable agent job state, distributed cache, or cross-service tracing exists.

For an interview, say what you would add: authentication tied to user data, TLS/network segmentation, secrets management, rate limits, request budgets/timeouts, schema migrations, health/readiness endpoints, OpenTelemetry tracing with correlation IDs, and policy controls around any financial advice output.

3.11 Test reality

The Java module has a single context-load smoke test with datasource excluded. The Python service has no test files, and there are no Java-to-Python or Compose integration tests. The local engine is used for manual comparisons, not automated test assertions.

A credible test plan:

1. Unit-test each indicator against a small OHLC fixture and known expected values, including insufficient rows, missing data, flat typical price, partial week, and split/adjusted data.
2. Contract-test FastAPI response schema and Java `QuantScoreResponse` deserialization.
3. Use WireMock/MockWebServer to test quant timeout/4xx/5xx and LangChain tool errors.
4. Add repository integration tests with PostgreSQL Testcontainers and query/index checks.
5. Add end-to-end Compose smoke tests with external providers mocked.
6. Evaluate LLM output with fixed golden cases and schema validation; avoid asserting prose verbatim.

3.12 Questions to rehearse

Why not put the quant math in Java?

Python let the computation use the market-data and analytics ecosystem directly, while Java was already the natural home for the persisted REST application and LangChain4j orchestration. The REST boundary keeps the contract explicit and allows independent deployment. It also creates failure modes, so I added a degraded response path and would add timeouts, circuit breaking, and contract tests next.

Explain the composite score to a non-finance engineer.

It is a simple weighted rule system, not a predictive model. Trend alignment contributes up to four points, momentum confirmation up to three, and CCI confirmation up to three. The sum is a transparent feature sent to the LLM alongside raw signals. It makes the numerical input inspectable instead of asking the LLM to derive indicators from prose.

Why drop the partial weekly candle?

A week in progress contains incomplete price/volume information. Including it produces a moving target and makes weekly indicators change intraperiod. Dropping it gives calculations a consistent set of completed weekly bars.

What if the Python service is down?

The Java client retries twice and returns an explicit unavailable quant DTO if it still fails. The prompt directs the agent to disclose that missing context and continue with available market tools. I would add an explicit HTTP timeout, circuit breaker, observability, and an API-level degraded flag for clients.

What is the weakness of tool-using LLM output?

Tool results can be missing, stale, malformed, or contradict one another, and LLM JSON can violate the schema. The project has an evaluator/repair loop and a validator, but the current prompt-validator mismatch shows why one shared schema and automated contract tests are necessary.

How would you scale trending scans?

`/agent/scan/trending` should become an asynchronous job: enqueue tickers, set concurrency/rate caps per vendor, persist job state and per-ticker results, deduplicate recent analyses, and notify/poll for completion. This prevents a long fan-out request from monopolizing one web thread or exceeding vendor limits.

3.13 Polyglot “implemented vs next” summary

Implemented	Next production step
Dockerized Java/Python/Postgres topology	Java readiness check, images pinned/scanned, deployment manifests
FastAPI quant contract and ticker validation	Versioned OpenAPI contract, timeout, fixtures, service integration tests
Transparent SMA/MACD/RSI/CCI score	One canonical robust engine, zero-denominator handling, backtesting/evaluation
LangChain4j tools and prompt context	Shared output schema, tool policy, tracing, model fallback and budgets
Degraded quant response and retries	Circuit breaker, bulkhead, async jobs, error classification
JPA persistence	Flyway migrations, indexes, user authorization
React dashboard	Dockerized production bundle and non-hardcoded API configuration

4. Cross-Project System-Design Questions

4.1 How do the projects handle external dependency failure differently?

The LLM Gateway separates concerns at a provider adapter layer and has a cache-first path. Its current provider selection can choose an alternate configured provider if the primary is not healthy by its configuration test, but it does not retry after a live HTTP failure. The polyglot service treats the Python scorer as optional context: it retries and passes an unavailable object to the prompt so the agent can continue. Both need explicit timeouts, dependency health based on live signals, circuit breakers, bounded retries, and observability before production.

4.2 When is caching unsafe?

Cache only responses that have a clear identity and acceptable staleness. In the gateway, exact caching includes model and generation parameters. Semantic caching is risky for user-specific, safety-sensitive, tool-using, or high-temperature output because semantic similarity does not imply identical intent. In the market system, market data is time-sensitive; cache keys need ticker, interval, source, as-of period, and a short TTL. A cache must never conceal an unavailable/upstream-error response as a valid result without explicit metadata.

4.3 What is the difference between a retry, timeout, fallback, and circuit breaker?

- **Timeout:** bounds one waiting period.
- **Retry:** repeats a request after a likely transient failure; must be bounded and idempotency-aware.
- **Fallback:** chooses an alternate provider/result path after failure.
- **Circuit breaker:** temporarily stops calls to a repeatedly failing dependency, fast-fails or uses a fallback, then probes recovery.

The gateway has adapter timeouts and configuration-health fallback but not a real circuit breaker. The polyglot system has retry and semantic degraded fallback but no configured HTTP timeout or breaker on the quant client.

4.4 What would you monitor first?

For LLM Gateway: end-to-end latency by provider/model/cache result, provider error/timeout rates, exact/semantic cache hit rates, semantic response quality feedback, rate-limit blocks, auth failures, and Redis/Postgres availability.

For Polyglot: quant-service latency/error rate, yfinance/data freshness, score distribution and NaN count, model/tool latency and failure rate, validation/repair-loop rate, recommendation persistence errors, and external-vendor quota exhaustion.

4.5 How would you make either service multi-tenant?

Authenticate identity at the edge, propagate tenant/user context, scope every query/cache key/rate-limit bucket, enforce authorization at data access, isolate secrets/configuration, and emit tenant-safe metrics/audits. For the LLM gateway, `apiKeyId` should be attributed to usage records and cache policy. For FinSight, every portfolio/income/expense/recommendation query must be constrained to the authenticated user, not merely linked with a foreign key.

5. Rapid-Review Answer Sheet

Five exact facts worth memorizing: LLM Gateway

1. The edge route is `POST /v1/chat/completions`; the cache order is Redis exact, then pgvector semantic, then provider.
2. Exact cache key is SHA-256 of canonical request JSON; exact Redis lookups are O(1) average.
3. The limiter uses a Redis ZSET and one atomic Lua script: evict, count, add, expire.
4. Semantic similarity is `1 - cosine_distance`; the pgvector embedding dimension is 1536 and threshold defaults to 0.95.
5. Current HTTP provider failure does not fail over; only an unhealthy/config-missing provider triggers alternate selection.

Five exact facts worth memorizing: Polyglot system

1. Java calls Python at `http://python-scoring-service:8000` over the Docker Compose network.
2. Python scores completed weekly candles from ten years of adjusted yfinance data; partial current weeks are dropped.
3. Served score = trend (±4) + momentum (-3/+1/+3) + CCI (±3), theoretical range -10 to +10.
4. Quant outage returns an “unavailable” payload and the LLM continues with tools; it is degraded mode, not a circuit breaker.
5. The FastAPI path uses `analysis_engine.py`; newer/robust local scoring code is not wired to the route.

Source map for self-study

LLM Gateway

Concern	Code location
Orchestration	`llm-gateway/src/main/java/com/llmgateway/service/ChatCompletionService.java`
Router/failover	`llm-gateway/src/main/java/com/llmgateway/provider/ProviderRouter.java`
Semantic SQL	`llm-gateway/src/main/java/com/llmgateway/cache/SemanticResponseCache.java`
Rate-limit Lua	`llm-gateway/src/main/java/com/llmgateway/ratelimit/RedisSlidingWindowRateLimiter.java`
Security filter	`llm-gateway/src/main/java/com/llmgateway/security/GatewaySecurityFilter.java`
Migrations	`llm-gateway/src/main/resources/db/migration/`
Config	`llm-gateway/src/main/resources/application.yml`
Tests	`llm-gateway/src/test/java/com/llmgateway/`

Polyglot system

Concern	Code location
Compose topology	`polyglot-system/docker-compose.yml`
FastAPI routes	`polyglot-system/apps/finance-vibe/src/finance_vibe/main.py`
Scoring API	`polyglot-system/apps/finance-vibe/src/finance_vibe/scoring_service.py`

Concern	Code location
Served engine	`polyglot-system/apps/finance-vibe/src/finance_vibe/analysis_engine.py`
Shadow engine	`polyglot-system/apps/finance-vibe/src/finance_vibe/analysis_engine_local.py`
Java agent	`polyglot-system/apps/financeAIAnalyzer/finance-ai/src/main/java/.../ai/AgentService.java`
Quant REST client	`polyglot-system/apps/financeAIAnalyzer/finance-ai/src/main/java/.../quant/QuantitativeScoringClient.java`
Prompt template	`polyglot-system/apps/financeAIAnalyzer/finance-ai/src/main/resources/prompts/stock_analyzer_prompt.txt`
React UI	`polyglot-system/apps/financeAIAnalyzer/finance-ai-frontend/`

Final interview reminder

The strongest answer is precise and falsifiable: say what path runs today, name the trade-off, and identify the next reliability/security/test improvement. If you do not know a number, do not invent one. If asked about a gap, explain how you discovered it and how you would test the fix.